

Advanced machine learning

Theory and generative models

Mikkel N. Schmidt

Technical University of Denmark,
DTU Compute, Department of Applied Mathematics and Computer Science.

Overview

- ① Module overview
- ② Results from last week's competition
- ③ Convolutions and Fourier transform
 - Discrete convolutions
 - Discrete Fourier transform
 - Graph convolutions and Fourier transform
- ④ GNNs and probabilistic models
- ⑤ Generative models
 - Traditional methods
 - Variational autoencoders
 - Generative adversarial networks (GANs)
 - Evaluating graph generation
- ⑥ Exercises

Module overview

Module 3: Graph models

1. *Graphs & node embeddings*

Reading: Hamilton ch. 1–4

Exercise 1

2. *Graph neural networks*

Reading: Hamilton ch. 5–6

Exercise 2

3. *Theory & generative models*

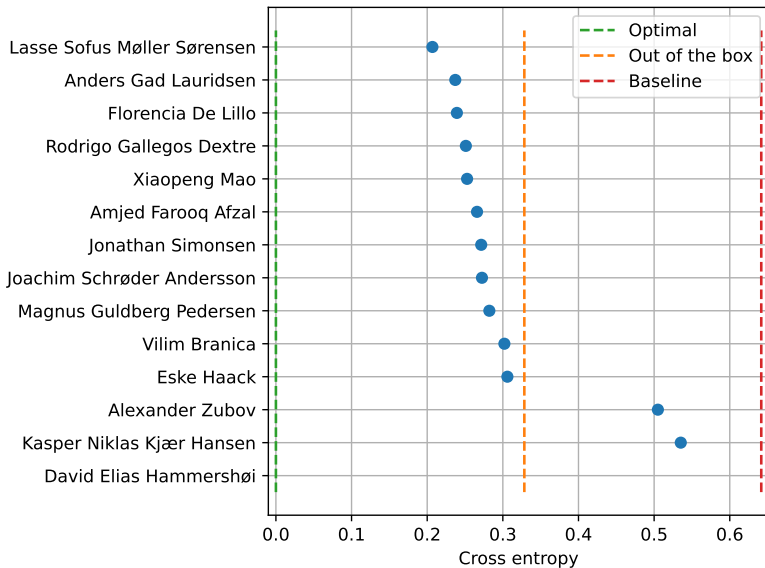
Reading: Hamilton ch. 7–9

Exercise 3

4. *Mini project 3*

Results from last week's competition

Last week's competition



Convolutions and Fourier transform

Convolutions and Fourier transform: Discrete convolutions

Convolution

- Convolution of discrete time series

$$y[n] = (x * h)[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k]$$

- Convolution of continuous signals

$$g(\mathbf{x}) = (f * h)(\mathbf{x}) = \int_{\mathbb{R}^d} f(\mathbf{y})h(\mathbf{x} - \mathbf{y})d\mathbf{y}$$

We only look at discrete convolutions in this course.

Causal filter

- When the filter coefficients are zero for $n < 0$, i.e. $h[n] = 0 \forall n < 0$, we say the filter is causal.

This means that the filter does not respond before the input signal is applied.

$$y[n] = (x * h)[n] = \sum_{k=-\infty}^n x[k]h[n-k] = \sum_{k'=0}^{\infty} x[n-k']h[k']$$

Causal filter

- When the filter coefficients are zero for $n < 0$, i.e. $h[n] = 0 \forall n < 0$, we say the filter is causal.

This means that the filter does not respond before the input signal is applied.

$$y[n] = (x * h)[n] = \sum_{k=-\infty}^n x[k]h[n-k] = \sum_{k'=0}^{\infty} x[n-k']h[k']$$

- We can then write the convolution as

$$y[n] = h[0]x[n] + h[1]x[n-1] + h[2]x[n-2] + \dots$$

This shows how the convolution is a weighted linear combination of delayed versions of the input signal.

Causal filter

- When the filter coefficients are zero for $n < 0$, i.e. $h[n] = 0 \forall n < 0$, we say the filter is causal.

This means that the filter does not respond before the input signal is applied.

$$y[n] = (x * h)[n] = \sum_{k=-\infty}^n x[k]h[n-k] = \sum_{k'=0}^{\infty} x[n-k']h[k']$$

- We can then write the convolution as

$$y[n] = h[0]x[n] + h[1]x[n-1] + h[2]x[n-2] + \dots$$

This shows how the convolution is a weighted linear combination of delayed versions of the input signal.

- If we define a unit delay operator

$$Dx[n] = x[n-1]$$

we can further write it as

$$y[n] = (h[0] + h[1]D + h[2]D^2 + \dots)x[n]$$

This is a nice parallel to graph convolutions as we will see later.

Circular convolution

- Circular discrete-time convolution

$$y[n] = (x \circledast h)[n] = \sum_{k=0}^{N-1} x[k] \cdot h[(n - k) \bmod N]$$

- We can define a (Toeplitz structured) filter matrix

$$\mathbf{H} = \begin{bmatrix} h[0] & h[N-1] & h[N-2] & \cdots & h[1] \\ h[1] & h[0] & h[N-1] & \cdots & h[2] \\ h[2] & h[1] & h[0] & \cdots & h[3] \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ h[N-1] & h[N-2] & h[N-3] & \cdots & h[0] \end{bmatrix} \quad \text{i.e. } H_{kn} = h[(n - k) \bmod N]$$

a compute the convolution as

$$\mathbf{y} = \mathbf{H}\mathbf{x}$$

Convolutions and Fourier transform: Discrete Fourier transform

Discrete Fourier transform

- The discrete Fourier transform is defined as

$$\tilde{x}[k] = \sum_{n=0}^{N-1} x[n] \overbrace{e^{-i \frac{2\pi k}{N} n}}^{\cos\left(\frac{2\pi kn}{N}\right) - i \sin\left(\frac{2\pi kn}{N}\right)} = \sum_{n=0}^{N-1} x[n] \underbrace{\omega_N^{k \cdot n}}_{\omega_N = e^{-i \frac{2\pi}{N}}}$$

Discrete Fourier transform

- The discrete Fourier transform is defined as

$$\tilde{x}[k] = \sum_{n=0}^{N-1} x[n] \overbrace{e^{-i \frac{2\pi k n}{N}}}^{\cos\left(\frac{2\pi k n}{N}\right) - i \sin\left(\frac{2\pi k n}{N}\right)} = \sum_{n=0}^{N-1} x[n] \underbrace{\omega_N^{k \cdot n}}_{\omega_N = e^{-i \frac{2\pi}{N}}}$$

- We can define a DFT basis matrix

$$U^H = \frac{1}{\sqrt{N}} \begin{bmatrix} \omega_N^{0 \cdot 0} & \omega_N^{0 \cdot 1} & \dots & \omega_N^{0 \cdot (N-1)} \\ \omega_N^{1 \cdot 0} & \omega_N^{1 \cdot 1} & \dots & \omega_N^{1 \cdot (N-1)} \\ \vdots & \vdots & \ddots & \vdots \\ \omega_N^{(N-1) \cdot 0} & \omega_N^{(N-1) \cdot 1} & \dots & \omega_N^{(N-1) \cdot (N-1)} \end{bmatrix} \quad \text{i.e. } U_{kn}^H = \omega_N^{k \cdot n}$$

The DFT basis is a unitary matrix, $U^H U = I$.

Discrete Fourier transform

- The discrete Fourier transform is defined as

$$\tilde{x}[k] = \sum_{n=0}^{N-1} x[n] \overbrace{e^{-i \frac{2\pi kn}{N}}}^{\cos\left(\frac{2\pi kn}{N}\right) - i \sin\left(\frac{2\pi kn}{N}\right)} = \sum_{n=0}^{N-1} x[n] \underbrace{\omega_N^{k \cdot n}}_{\omega_N = e^{-i \frac{2\pi}{N}}}$$

- We can define a DFT basis matrix

$$U^H = \frac{1}{\sqrt{N}} \begin{bmatrix} \omega_N^{0 \cdot 0} & \omega_N^{0 \cdot 1} & \dots & \omega_N^{0 \cdot (N-1)} \\ \omega_N^{1 \cdot 0} & \omega_N^{1 \cdot 1} & \dots & \omega_N^{1 \cdot (N-1)} \\ \vdots & \vdots & \ddots & \vdots \\ \omega_N^{(N-1) \cdot 0} & \omega_N^{(N-1) \cdot 1} & \dots & \omega_N^{(N-1) \cdot (N-1)} \end{bmatrix} \quad \text{i.e. } U_{kn}^H = \omega_N^{k \cdot n}$$

The DFT basis is a unitary matrix, $U^H U = I$.

- The DFT and inverse DFT can be computed as

$$\tilde{\mathbf{x}} = U^H \mathbf{x} \quad \mathbf{x} = U \tilde{\mathbf{x}}$$

DFT maps a signal onto a basis of complex sinusoids with different frequencies.

Filtering with the DFT

- Convolution in the “time” domain

$$y[n] = (x \circledast h)[n] = \sum_{k=0}^{N-1} x[k] \cdot h[(n - k) \bmod N]$$

corresponds to multiplication in the “frequency” domain

$$\tilde{y}[k] = \tilde{x}[k] \cdot \tilde{h}[k]$$

Filtering with the DFT

- Convolution in the “time” domain

$$y[n] = (x \circledast h)[n] = \sum_{k=0}^{N-1} x[k] \cdot h[(n-k) \bmod N]$$

corresponds to multiplication in the “frequency” domain

$$\tilde{y}[k] = \tilde{x}[k] \cdot \tilde{h}[k]$$

- In matrix form

$$\mathbf{y} = \mathbf{H}\mathbf{x} \quad \Leftrightarrow \quad \tilde{\mathbf{y}} = \tilde{\mathbf{h}} \odot \tilde{\mathbf{x}}$$

i.e.

$$\mathbf{y} = \mathbf{U}(\underbrace{\mathbf{U}^H \mathbf{h}}_{\tilde{\mathbf{h}} = \text{DFT}(\mathbf{h})} \odot \underbrace{\mathbf{U}^H \mathbf{x}}_{\tilde{\mathbf{x}} = \text{DFT}(\mathbf{x})})$$

Proof: Convolution equals DFT multiplication

$$\tilde{y}[k] = \sum_{n=0}^{N-1} y[n] \omega_N^{k \cdot n}$$

Proof: Convolution equals DFT multiplication

$$\begin{aligned}\tilde{y}[k] &= \sum_{n=0}^{N-1} y[n] \omega_N^{k \cdot n} \\ &\quad \underbrace{y[n] = (x \circledast h)[n]} \\ &= \sum_{n=0}^{N-1} \sum_{\ell=0}^{N-1} x[\ell] \cdot h[(n - \ell) \bmod N] \omega_N^{k \cdot n}\end{aligned}$$

Proof: Convolution equals DFT multiplication

$$\begin{aligned}\tilde{y}[k] &= \sum_{n=0}^{N-1} y[n] \omega_N^{k \cdot n} \\ &= \underbrace{\sum_{n=0}^{N-1} \sum_{\ell=0}^{N-1} x[\ell] \cdot h[(n - \ell) \bmod N]}_{y[n] = (x \circledast h)[n]} \omega_N^{k \cdot n} \\ &= \sum_{\ell=0}^{N-1} x[\ell] \sum_{n=0}^{N-1} h[(n - \ell) \bmod N] \omega_N^{k \cdot n}\end{aligned}$$

Proof: Convolution equals DFT multiplication

$$\begin{aligned}
 \tilde{y}[k] &= \sum_{n=0}^{N-1} y[n] \omega_N^{k \cdot n} \\
 &\quad \underbrace{y[n] = (x \circledast h)[n]} \\
 &= \sum_{n=0}^{N-1} \sum_{\ell=0}^{N-1} x[\ell] \cdot h[(n - \ell) \bmod N] \omega_N^{k \cdot n} \\
 &= \sum_{\ell=0}^{N-1} x[\ell] \sum_{n=0}^{N-1} h[(n - \ell) \bmod N] \omega_N^{k \cdot n} \\
 &= \sum_{\ell=0}^{N-1} x[\ell] \sum_{m=-\ell}^{N-1-\ell} h[m \bmod N] \omega_N^{k \cdot \overbrace{(m+\ell)}^{n=m+\ell}}
 \end{aligned}$$

Proof: Convolution equals DFT multiplication

$$\begin{aligned}
 \tilde{y}[k] &= \sum_{n=0}^{N-1} y[n] \omega_N^{k \cdot n} \\
 &\quad \underbrace{y[n] = (x \circledast h)[n]} \\
 &= \sum_{n=0}^{N-1} \sum_{\ell=0}^{N-1} x[\ell] \cdot h[(n - \ell) \bmod N] \omega_N^{k \cdot n} \\
 &= \sum_{\ell=0}^{N-1} x[\ell] \sum_{n=0}^{N-1} h[(n - \ell) \bmod N] \omega_N^{k \cdot n} \\
 &\quad \quad \quad n = m + \ell \\
 &= \sum_{\ell=0}^{N-1} x[\ell] \sum_{m=-\ell}^{N-1-\ell} h[m \bmod N] \omega_N^{k \cdot \overbrace{(m+\ell)}^{\ell}} \\
 &= \sum_{\ell=0}^{N-1} x[\ell] \underbrace{\sum_{m=0}^{N-1} h[m] \omega_N^{k \cdot m} \omega_N^{k \cdot \ell}}_{\tilde{h}[k]}
 \end{aligned}$$

Proof: Convolution equals DFT multiplication

$$\begin{aligned}
 \tilde{y}[k] &= \sum_{n=0}^{N-1} y[n] \omega_N^{k \cdot n} \\
 &\quad \underbrace{y[n] = (x \circledast h)[n]} \\
 &= \sum_{n=0}^{N-1} \sum_{\ell=0}^{N-1} x[\ell] \cdot h[(n - \ell) \bmod N] \omega_N^{k \cdot n} \\
 &= \sum_{\ell=0}^{N-1} x[\ell] \sum_{n=0}^{N-1} h[(n - \ell) \bmod N] \omega_N^{k \cdot n} \\
 &= \sum_{\ell=0}^{N-1} x[\ell] \sum_{m=-\ell}^{N-1-\ell} h[m \bmod N] \omega_N^{k \cdot \overbrace{(m+\ell)}^{n=m+\ell}} \\
 &= \sum_{\ell=0}^{N-1} x[\ell] \underbrace{\sum_{m=0}^{N-1} h[m] \omega_N^{k \cdot m}}_{\tilde{h}[k]} \omega_N^{k \cdot \ell} \\
 &= \sum_{\ell=0}^{N-1} x[\ell] \tilde{h}[k] \omega_N^{k \cdot \ell}
 \end{aligned}$$

Proof: Convolution equals DFT multiplication

$$\begin{aligned}
 \tilde{y}[k] &= \sum_{n=0}^{N-1} y[n] \omega_N^{k \cdot n} \\
 &\quad \underbrace{y[n] = (x \circledast h)[n]} \\
 &= \sum_{n=0}^{N-1} \sum_{\ell=0}^{N-1} x[\ell] \cdot h[(n - \ell) \bmod N] \omega_N^{k \cdot n} \\
 &= \sum_{\ell=0}^{N-1} x[\ell] \sum_{n=0}^{N-1} h[(n - \ell) \bmod N] \omega_N^{k \cdot n} \\
 &= \sum_{\ell=0}^{N-1} x[\ell] \sum_{m=-\ell}^{N-1-\ell} h[m \bmod N] \omega_N^{k \cdot \overbrace{(m+\ell)}^{n=m+\ell}} \\
 &= \sum_{\ell=0}^{N-1} x[\ell] \underbrace{\sum_{m=0}^{N-1} h[m] \omega_N^{k \cdot m}}_{\tilde{h}[k]} \omega_N^{k \cdot \ell} \\
 &= \sum_{\ell=0}^{N-1} x[\ell] \tilde{h}[k] \omega_N^{k \cdot \ell} = \underbrace{\sum_{\ell=0}^{N-1} x[\ell] \omega_N^{k \cdot \ell}}_{\tilde{x}[k]} \tilde{h}[k]
 \end{aligned}$$

Proof: Convolution equals DFT multiplication

$$\begin{aligned}
 \tilde{y}[k] &= \sum_{n=0}^{N-1} y[n] \omega_N^{k \cdot n} \\
 &\quad \underbrace{y[n] = (x \circledast h)[n]} \\
 &= \sum_{n=0}^{N-1} \sum_{\ell=0}^{N-1} x[\ell] \cdot h[(n - \ell) \bmod N] \omega_N^{k \cdot n} \\
 &= \sum_{\ell=0}^{N-1} x[\ell] \sum_{n=0}^{N-1} h[(n - \ell) \bmod N] \omega_N^{k \cdot n} \\
 &= \sum_{\ell=0}^{N-1} x[\ell] \sum_{m=-\ell}^{N-1-\ell} h[m \bmod N] \omega_N^{k \cdot \overbrace{(m+\ell)}^{n=m+\ell}} \\
 &= \sum_{\ell=0}^{N-1} x[\ell] \underbrace{\sum_{m=0}^{N-1} h[m] \omega_N^{k \cdot m}}_{\tilde{h}[k]} \omega_N^{k \cdot \ell} \\
 &= \sum_{\ell=0}^{N-1} x[\ell] \tilde{h}[k] \omega_N^{k \cdot \ell} = \underbrace{\sum_{\ell=0}^{N-1} x[\ell] \omega_N^{k \cdot \ell}}_{\tilde{x}[k]} \tilde{h}[k] = \tilde{x}[k] \cdot \tilde{h}[k]
 \end{aligned}$$

DFT basis

■ DFT basis vectors

$$\mathbf{u}_k^* = [\omega_N^{k \cdot 0}, \omega_N^{k \cdot 1}, \dots, \omega_N^{k \cdot (N-1)}]^\top$$

are complex exponentials with normalized frequency $\frac{k}{N}$

$$(\mathbf{u}_k^*)_n = \omega_N^{k \cdot n} = e^{-i \frac{2\pi k}{N} n} = \cos\left(\frac{2\pi k}{N} n\right) - i \sin\left(\frac{2\pi k}{N} n\right)$$

DFT basis

- DFT basis vectors

$$\mathbf{u}_k^* = [\omega_N^{k \cdot 0}, \omega_N^{k \cdot 1}, \dots, \omega_N^{k \cdot (N-1)}]^\top$$

are complex exponentials with normalized frequency $\frac{k}{N}$

$$(\mathbf{u}_k^*)_n = \omega_N^{k \cdot n} = e^{-i \frac{2\pi k}{N} n} = \cos\left(\frac{2\pi k}{N} n\right) - i \sin\left(\frac{2\pi k}{N} n\right)$$

- Come in complex conjugate pairs

$$(\mathbf{u}_{N-k}^*)_n = e^{-i \frac{2\pi(N-k)}{N} n} = e^{-i \frac{2\pi N}{N} n} \overbrace{e^{i \frac{2\pi k}{N} n}}^{=1} = e^{i \frac{2\pi k}{N} n} = (\mathbf{u}_k)_n \quad (1)$$

i.e. there are two complex conjugate components per frequency.

Convolutions and Fourier transform: Graph convolutions and Fourier transform

Graph convolutions

- For a discrete time signal, we could write a convolution as a weighted sum of delayed versions of the input signal.

$$y[n] = (h[0] + h[1]D + h[2]D^2 + \cdots)x[n]$$

Here, the shift operator is the unit delay $Dx[n] = x[n - 1]$. On a chain graph, it would correspond to moving on to the next node (e.g. clockwise).

Graph convolutions

- For a discrete time signal, we could write a convolution as a weighted sum of delayed versions of the input signal.

$$y[n] = (h[0] + h[1]D + h[2]D^2 + \cdots)x[n]$$

Here, the shift operator is the unit delay $Dx[n] = x[n - 1]$. On a chain graph, it would correspond to moving on to the next node (e.g. clockwise).

- For a signal propagating on a graph, we can use the adjacency matrix as a “neighborhood shift” operator.

Thus we can define a graph convolution as:

$$\mathbf{y} = (h[0]\mathbf{I} + h[1]\mathbf{A} + h[2]\mathbf{A}^2 + \cdots)\mathbf{x}$$

Eigendecomposition of the cycle graph adjacency matrix

- Eigenvalue decomposition is given by

$$\mathbf{A} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^H$$

where the eigenvectors satisfy

$$\mathbf{A}\mathbf{u}_k = \lambda_k\mathbf{u}_k$$

Eigendecomposition of the cycle graph adjacency matrix

- Eigenvalue decomposition is given by

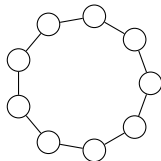
$$\mathbf{A} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^H$$

where the eigenvectors satisfy

$$\mathbf{A}\mathbf{u}_k = \lambda_k\mathbf{u}_k$$

- It turns out that the eigenvectors are equal to the *DFT basis vectors*.

You will show this in the exercises.



$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

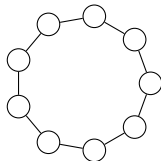
Eigendecomposition of the cycle graph adjacency matrix

- Eigenvalue decomposition is given by

$$\mathbf{A} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^H$$

where the eigenvectors satisfy

$$\mathbf{A}\mathbf{u}_k = \lambda_k\mathbf{u}_k$$



- It turns out that the eigenvectors are equal to the *DFT basis vectors*.

You will show this in the exercises.

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

Note: The eigendecomposition not unique

- Conjugate pairs can be rotated together.
- We can rotate to sine and cosine components, yielding a real matrix $\mathbf{U}$.

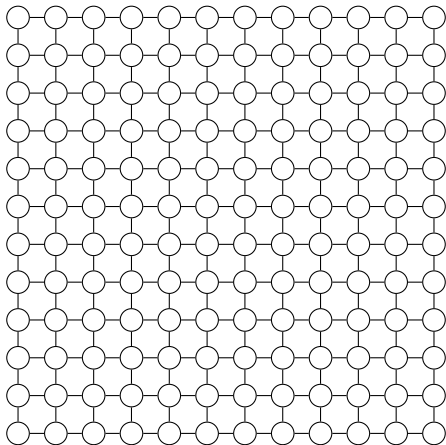
Graph Fourier transform

- We can now define a generalized *graph Fourier transform*, given as the eigendecomposition of the adjacency matrix.

$$A = U\Lambda U^H$$

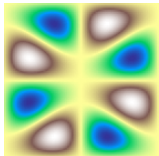
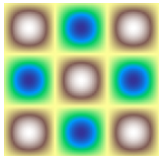
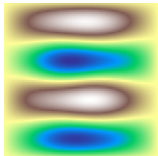
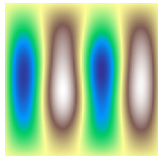
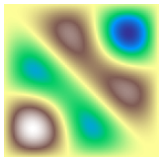
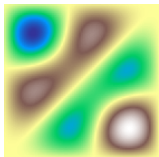
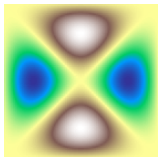
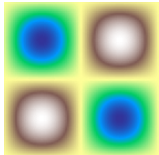
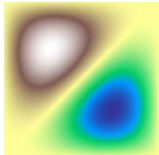
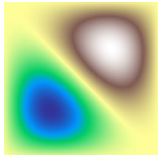
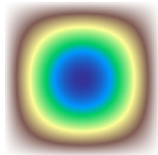
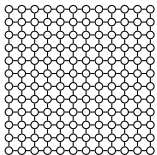
- The basis vectors are no longer *frequencies* but more general *spatial patterns* on the graph vertices.

Example: Grid graph



Example: Spatial components of the grid graph

Twelve “low-frequency” components.



Summary of graph covolution

- A graph convolution is defined as

$$\mathbf{y} = \sum_{k=0}^N h[k] \mathbf{A}^k \mathbf{x}$$

- It can equivalently be computed using the graph Fourier transform

$$\mathbf{y} = \mathbf{U} \left(\sum_{k=0}^N h[k] \mathbf{\Lambda}^k \right) \mathbf{U}^H \mathbf{x}$$

where

$$\mathbf{A} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^H$$

GNNs without message passing

- We can now define a graph convolution neural network

$$\widehat{\mathbf{Z}} = \text{MLP}_{\text{out}} \left(\underbrace{\sum_{k=0}^N h[k] \mathbf{A}^k}_{\text{Graph convolution}} \overbrace{\text{MLP}_{\text{in}}(\mathbf{X})}^{\text{MLP on node input features}} \right)$$

Node-level output features

Computations can also be performed in the Fourier domain.

- No message passing, but information flows through N steps from nodes to neighbors through the graph convolution.

GNNs and probabilistic models

Probabilistic graphical models

- *Probabilistic graphical models*: Represent statistical relationships between random variables using a graph.
- *Approximate inference*: Finding the exact distributions of hidden variables is difficult.
- *Message passing*: Iteratively pass messages between nodes that updates beliefs about node's distribution based on neighbors.
- *Variational approach*: Using a simpler, approximate distribution for each variable.
- *Conjugate models*: Classical variational message passing uses conjugate distributions so updates can be performed exactly.

Kernel mean embedding

- Probability distributions can be embedded in a Hilbert space.
Hilbert space = A complete vector space with an inner product.
- Mean embedding using a feature map ϕ

$$\mu_x = \int_{\mathbb{R}^d} \phi(\mathbf{x}) p(\mathbf{x}) d\mathbf{x}$$

If the feature map is “complex enough” the embedding is injective

- In practice, we do not work directly with the feature map, but define it implicitly using a kernel (inner product).

$$\langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle = k(\mathbf{x}, \mathbf{x}')$$

Graphs as graphical models

- A graph defines a Markov random field

$$p(\overbrace{\{x_v\}}^{\text{Node features (observed)}}, \underbrace{\{z_v\}}_{\text{Node embedding (latent variables)}}) \propto \prod_{v \in \mathcal{V}} \Phi(x_v, z_v) \prod_{(u,v) \in \mathcal{E}} \Psi(z_u, z_v)$$

Graphs as graphical models

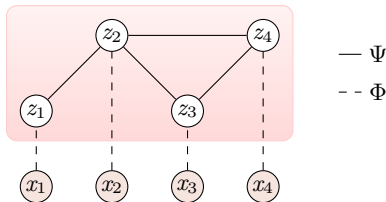
- A graph defines a Markov random field

Node features (observed)

$$p(\underbrace{\{x_v\}}_{\text{Node embedding (latent variables)}}, \underbrace{\{z_v\}}_{\text{Node features (observed)}}) \propto \prod_{v \in \mathcal{V}} \Phi(x_v, z_v) \prod_{(u,v) \in \mathcal{E}} \Psi(z_u, z_v)$$

Node embedding (latent variables)

- Example



Nodes could be vector-valued, but I will assume scalars for simplicity.

Mean field variational inference

- Approximate posterior distribution of latent variables

$$p(\{z_v\}|\{x_v\}) \approx q(\{z_v\}) = \prod_{v \in \mathcal{V}} q_v(z_v)$$

- Minimize KL to true posterior

$$\text{KL}(q|p) = \int \prod_{v \in \mathcal{V}} q_v(z_v) \log \frac{\prod_{v \in \mathcal{V}} q_v(z_v)}{p(\{z_v\}|\{x_v\})} \prod_{v \in \mathcal{V}} \text{d}z_v$$

- Optimum satisfies fixed point equations

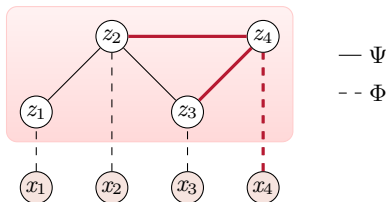
$$\log q_v^{(t+1)}(z_v) = c_v + \log \Phi(x_v, z_v) + \sum_{u \in \mathcal{N}(v)} \int q_u^{(t)}(z_u) \log \Psi(z_u, z_v) \text{d}z_u$$

Mean field variational message passing

- Optimum satisfies fixed point equations

$$\log q_v^{(t+1)}(z_v) = c_v + \log \Phi(x_v, z_v) + \sum_{u \in \mathcal{N}(v)} \int q_u^{(t)}(z_u) \log \Psi(z_u, z_v) dz_u$$

- Example (node 4)



$$\log q_4^{(t)}(z_4) = c_4 + \log \Phi(x_4, z_4) + \int q_2^{(t)}(z_2) \log \Psi(z_2, z_4) dz_2 + \int q_3^{(t)}(z_3) \log \Psi(z_3, z_4) dz_3$$

Mean field VI embedded in Hilbert space

- Suppose we represent $q_v(z_v)$ as mean embeddings

$$\mu_v = \int \phi(z) q_v(z_v) dz$$

the fixed point equations can be written as

$$\mu_v^{(t+1)} = c + f\left(\mu_v^{(t)}, x_v, \{\mu_u, \forall u \in \mathcal{N}(v)\}\right)$$

This looks very much like message passing GNN updates.

- We can now use this to either
 - Define potential functions Φ and Ψ and a kernel $k(\cdot, \cdot)$ and analytically derive updates.
 - Learn embeddings and assume that they *could* correspond to some probabilistic model.

Generative models

Generative models: Traditional methods

Traditional models

- *Erdős-Rényi model*

Generate each possible edge independently with a fixed probability

$$P(A_{u,v} = 1) = r, \quad \forall u, v \in \mathcal{V}, u \neq v.$$

Traditional models

■ *Erdős-Rényi model*

Generate each possible edge independently with a fixed probability

$$P(A_{u,v} = 1) = r, \quad \forall u, v \in \mathcal{V}, u \neq v.$$

■ *Stochastic blockmodels*

1. Specify a categorical distribution $p_1, \dots, p_\gamma$ over blocks $\mathcal{C}_1, \dots, \mathcal{C}_\gamma$.
2. Assign each node randomly to a block according to p_i .
3. Specify probabilities of edges between (and within) each block $r_{i,j}$
4. Generate edges between nodes in blocks i and j according to

$$P(A_{u,v} = 1) = r_{i,j}, \quad \forall u \in \mathcal{C}_i, v \in \mathcal{C}_j, u \neq v, \forall i, j \in (1, \dots, \gamma)$$

Possible simplification

$$r_{i,j} = \begin{cases} r_{\text{in}} & i = j \\ r_{\text{out}} & i \neq j \end{cases}$$

Traditional models

■ *Preferential attachment*

1. Start with a seed graph, e.g. a fully connected graph with m_0 nodes
(Possibly just a single node)
2. Iteratively add nodes to the graph and form edges according to

$$P(A_{u,v} = 1) = \frac{d_v^{(t)}}{\sum_{v' \in \mathcal{V}^{(t)}} d_v^{(t)}}$$

Nodes that are already highly connected are more likely to connect.

There are also linear and non-linear extensions that allows more detailed control of the distribution.

Applications of traditional methods

- *Generating synthetic benchmark data:* Testing algorithms, validating hypotheses, and studying network phenomena in controlled settings.
- *Creating null models:* If observed properties of a real graph significantly differ from those of the null model, it suggests that the real graph exhibits non-random structural characteristics or underlying processes.

Generative models: Variational autoencoders

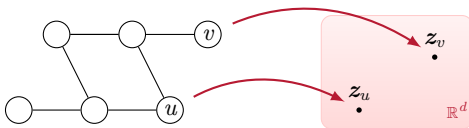
Latent variable models

- We have already worked with a generative model: *Shallow node embedding*.
- Each node is embedded in a space, i.e. is endowed with a latent variable.
- The graph edges are generated according to a pairwise decoder.

Encoder-decoder perspective

Encoder: Maps a vectex into a latent representation.

$$\text{ENC} : \mathcal{V} \rightarrow \mathbb{R}^d$$



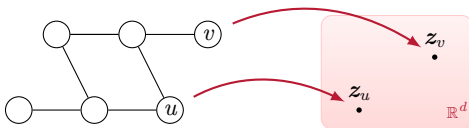
Example 1: Learned embedding (lookup table).

Example 2: Message passing graph neural network.

Encoder-decoder perspective

Encoder: Maps a vertex into a latent representation.

$$\text{ENC} : \mathcal{V} \rightarrow \mathbb{R}^d$$



Example 1: Learned embedding (lookup table).

Example 2: Message passing graph neural network.

Pairwise decoder: Maps a pair of latent variables into a node pair statistic.

$$\text{DEC} : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{S}$$

Example 1: Predict link or non-link, $\mathbb{S} = \{0, 1\}$.

Example 2: Predict a graph-based similarity measure, $\mathbb{S} = \mathbb{R}_+ = \{x \in \mathbb{R} | x \geq 0\}$.

Variational autoencoders

- *Probabilistic encoder*: Takes graph as input and computes parameters of a probabilistic latent embedding.

$$\mathbf{Z} \sim \mathcal{N}(\boldsymbol{\mu}_{\phi}(\mathcal{G}), \boldsymbol{\Sigma}_{\phi}(\mathcal{G}))$$

For example the mean and variance of a Normal distribution.

- *Probabilistic decoder*: Takes a latent representation and produces a conditional distribution over entries of the adjacency matrix.

$$\mathbf{A}_{u,v} \sim \text{Bernoulli}(p_{u,v}(\mathbf{Z}))$$

For example the parameters of independent Bernoulli variables for each possible link.

- *A prior distribution*:

$$p(\mathbf{Z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Typically this is a standard Normal.

Variational autoencoders

- *Probabilistic encoder:* $\mathbf{Z} \sim \mathcal{N}(\boldsymbol{\mu}_\phi(\mathcal{G}), \boldsymbol{\Sigma}_\phi(\mathcal{G}))$
- *Probabilistic decoder:* $\mathbf{A}_{u,v} \sim \text{Bernoulli}(p_{u,v}(\mathbf{Z}))$
- *A prior distribution:* $p(\mathbf{Z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$

Given these components we train the model by maximizing the evidence lower bound (ELBO)

$$\mathcal{L} = \sum_{\mathcal{G} \in \mathcal{D}} \mathbb{E}_{q_\phi(\mathbf{Z}|\mathcal{G})} [p_\theta(\mathcal{G}|\mathbf{Z})] - \text{KL} [q_\phi(\mathbf{Z}|\mathcal{G}) \parallel p(\mathbf{Z})]$$

Node-level latents

- *Encoder*: Probabilistic node embedding

GNN outputs two vectors per node

$$\underbrace{\boldsymbol{\mu}_Z, \log \sigma_Z}_{\text{Mean and variance for each node}} = \overbrace{\text{GNN}(\mathbf{A}, \mathbf{X})}$$

Mean and variance for each node

Sample latent node embedding using the reparametrization trick

$$\mathbf{Z} = \boldsymbol{\mu}_Z + \boldsymbol{\epsilon} \odot \log \sigma_Z$$

Node-level latents

- *Encoder*: Probabilistic node embedding

GNN outputs two vectors per node

$$\underbrace{\boldsymbol{\mu}_Z, \log \sigma_Z}_{\text{Mean and variance for each node}} = \overbrace{\text{GNN}(\mathbf{A}, \mathbf{X})}$$

Mean and variance for each node

Sample latent node embedding using the reparametrization trick

$$\mathbf{Z} = \boldsymbol{\mu}_Z + \boldsymbol{\epsilon} \odot \log \sigma_Z$$

- *Decoder*: Probabilistic pairwise decoder compute a binary probability

$$P(A_{u,v} = 1) = \sigma(\mathbf{z}_u^\top \mathbf{z}_v + \mathbf{b}n)$$

Same decoder we used for shallow embeddings.

Node-level latents

- *Encoder*: Probabilistic node embedding

GNN outputs two vectors per node

$$\underbrace{\boldsymbol{\mu}_Z, \log \sigma_Z}_{\text{Mean and variance for each node}} = \overbrace{\text{GNN}(\mathbf{A}, \mathbf{X})}^{\text{GNN outputs two vectors per node}}$$

Mean and variance for each node

Sample latent node embedding using the reparametrization trick

$$\mathbf{Z} = \boldsymbol{\mu}_Z + \boldsymbol{\epsilon} \odot \log \sigma_Z$$

- *Decoder*: Probabilistic pairwise decoder compute a binary probability

$$P(A_{u,v} = 1) = \sigma(\mathbf{z}_u^\top \mathbf{z}_v + \mathbf{b}n)$$

Same decoder we used for shallow embeddings.

Note: The basic node-level VAE has very limited performance, because the latent variable model (decoder) is very weak. This can be improved by using a GNN as decoder also.

Graph-level latents

- *Encoder*: Probabilistic graph embedding

GNN outputs two vectors

$$\underbrace{\mu_z, \log \sigma_z}_{\text{Mean and variance for the entire graph}} = \overbrace{\text{GNN}(\mathbf{A}, \mathbf{X})}^{\text{GNN outputs two vectors}}$$

Mean and variance for the entire graph

Sample latent graph embedding using the reparametrization trick

$$\mathbf{z} = \mu_z + \epsilon \odot \log \sigma_z$$

Graph-level latents

- *Encoder*: Probabilistic graph embedding

$$\underbrace{\mu_z, \log \sigma_z}_{\text{Mean and variance for the entire graph}} = \overbrace{\text{GNN}(\mathbf{A}, \mathbf{X})}^{\text{GNN outputs two vectors}}$$

Sample latent graph embedding using the reparametrization trick

$$\mathbf{z} = \mu_z + \epsilon \odot \log \sigma_z$$

- *Decoder*: Neural network

$$\underbrace{\tilde{\mathbf{A}}}_{\text{A matrix of edge probabilities}} = \overbrace{\sigma(\text{MLP}(\mathbf{z}))}^{\text{Multilayer perceptron}}$$

Graph-level latents

- *Encoder*: Probabilistic graph embedding

$$\underbrace{\mu_z, \log \sigma_z}_{\text{Mean and variance for the entire graph}} = \overbrace{\text{GNN}(\mathbf{A}, \mathbf{X})}^{\text{GNN outputs two vectors}}$$

Sample latent graph embedding using the reparametrization trick

$$\mathbf{z} = \mu_z + \epsilon \odot \log \sigma_z$$

- *Decoder*: Neural network

$$\underbrace{\tilde{\mathbf{A}}}_{\text{A matrix of edge probabilities}} = \overbrace{\sigma(\text{MLP}(\mathbf{z}))}^{\text{Multilayer perceptron}}$$

Note: Two problems:

1. *We need to assume a fixed number of nodes*: This can be handled by assuming a maximum number and masking.
2. *We do not know the order of the nodes*: This can be handled by (heuristic) graph matching to optimize the node order on the fly.

Generative models: Generative adversarial networks (GANs)

Adversarial approaches

- *Generator*: Generates data from a (random) latent variable

$$g_{\theta} : \mathbb{R}^d \rightarrow \mathcal{X}$$

- *Discriminator*: Distinguish between real and generated data

$$d_{\phi} : \mathcal{X} \rightarrow [0, 1]$$

- *Adversarial optimization*: Train the discriminator to distinguish real and generated, while training the generator to fool the discriminator

$$\min_{\theta} \max_{\phi} \underbrace{\mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [\log(1 - d_{\phi}(\mathbf{x}))]}_{\text{Sample from data distribution}} + \underbrace{\mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} [\log d_{\phi}(g_{\theta}(\mathbf{z}))]}_{\text{Sample from prior}}$$

GAN for graph generation

- *Generator*: Could be as simple multilayer perceptron.

$$\tilde{\mathbf{A}} = \sigma(\text{MLP}(\mathbf{z}))$$

- *Discriminator*: GNN based graph classification.

This approach does not require a specific node ordering: The discriminator is permutation invariant.

Generative models: Evaluating graph generation

Evaluating graph generation

- Difficult to directly compare generative models.
No uniform definition of likelihood across different methods.

Evaluating graph generation

- Difficult to directly compare generative models.
No uniform definition of likelihood across different methods.
- *Analyze graph statistics*: We can define a number of statistics such as the distributions of
 - Degree
 - Clustering coefficient
 - Eigenvector centrality

Measure a distributional distance between generated graphs and real test graphs.

Evaluating graph generation

- Difficult to directly compare generative models.
No uniform definition of likelihood across different methods.
- *Analyze graph statistics*: We can define a number of statistics such as the distributions of
 - Degree
 - Clustering coefficient
 - Eigenvector centrality

Measure a distributional distance between generated graphs and real test graphs.

- Distance measure: Total variation

$$d(s_{\text{gen}}, s_{\text{test}}) = \sup_x |P(s_{\text{gen}} \in x), P(s_{\text{test}} \in x)|$$

Measure the maximum disagreement in probability assignment. For discrete distributions this is simply half the L_1 distance.

Exercises

Exercises

- A Graph convolutions
- B Graph Fourier transform
- C Programming exercise (Graph Fourier transform)